

RECAP - Logik

- Aussagenlogik behandelt die logische Verknüpfung von Aussagen mittels Junktoren (z.B. und, oder, nicht, genau dann wenn)
- Begriffe: Elementare Aussagen, atomare Aussagen, Literale
- Mit Hilfe von Wahrheitstafeln können wir die Äquivalenz von Aussagen untersuchen.
- Wir können Aussagen in die Konjunktive Normalform und auch in die 3-Konjunktive Normalform umformen. (ebenso disjunktive Normalform)
- Eine Belegung erfüllt eine Aussage, wenn sie den Wahrheitswert 1 ergibt. Sie ist dann ein Modell der Aussage.
- Prädikatenlogik ergänzt um Existenz- und Allquantoren.



Grammatiken und die Chomsky Hierarchie

Grundbegriffe zu Sprachen

- Definition 1
 - Ein *Alphabet* ist eine beliebige *endliche, nichtleere* Menge. Die Elemente dieser Menge heißen *Zeichen*.

- Definition 2
 - Irgendeine (auch leere) Zeichenkette ist ein *Wort*. Man sagt: Eine Zeichenkette „ w ist ein Wort über dem Alphabet A “, wenn sämtliche Zeichen von w aus A stammen.
 - Das Symbol für das *leere Wort* ist ε .

Grundbegriffe zu Sprachen

- Definition 3

- Die *Länge eines Wortes* w , kurz: $|w|$, ist bestimmt durch die Anzahl aller Zeichen, die das Wort enthält. Dabei werden Mehrfachvorkommen entsprechend ihres Auftretens mehrfach gezählt.

- Definition 4

- Die Menge aller Wörter über A nennt man die *Wortmenge* A^* . Das leere Wort ε gehört zu jeder Wortmenge.
- $A^+ = A^* \setminus \{\varepsilon\}$

Grundbegriffe zu Sprachen

- Definition 5
 - Zwei Mengen $M1$ und $M2$ heißen *gleichmächtig*, wenn es eine bijektive Abbildung von $M1$ auf $M2$ gibt.
 - Ist eine der beiden Mengen die der natürlichen Zahlen $\mathbb{N}$, so ist M eine *abzählbar unendliche* Menge, kurz: „ M ist abzählbar“.

- Hinweis: die Menge der reellen Zahlen ist nicht abzählbar, kurz: überabzählbar unendlich

Grundbegriffe zu Sprachen

- Satz 1
 - Die Menge A^* aller Wörter über A ist abzählbar unendlich.

- Definition 6
 - Sei A ein Alphabet. Jede Teilmenge $L \subseteq A^*$ heißt *Sprache über A* .
 - (L erinnert an das englische Wort language für Sprache.)

Abbildungen

- Eine Funktion $f:X \rightarrow Y$ ist **injektiv**, wenn es zu jedem Element y der Zielmenge Y höchstens ein (also eventuell gar kein) Element x der Ausgangs- oder Definitionsmenge X gibt, das darauf zielt, wenn also nie zwei oder mehr verschiedene Elemente der Definitionsmenge auf dasselbe Element der Zielmenge abgebildet werden:

$$f(x_1)=f(x_2) \Rightarrow x_1=x_2$$

- Die Zielmenge kann daher nicht weniger mächtig als die Definitionsmenge sein, d. h., sie kann nicht weniger Elemente enthalten.
- Die Bildmenge $f(X):=\{f(x) \mid x \in X\}$ darf eine echte Teilmenge der Zielmenge Y sein, d. h., es kann Elemente y in Y geben, die keine Bildelemente $f(x)$ sind. Dies macht den Unterschied zu einer **bijektiven** Abbildung aus, von der außer Injektivität noch verlangt wird, dass jedes Element der Zielmenge als Bildelement $f(x)$ auftritt, dass also f **surjektiv** ist.

Formale Grammatiken

- Mathematische Modelle von Grammatiken, die zur eindeutigen Erzeugung und Beschreibung formaler Sprachen dienen.
- Einsatz in der theoretischen Informatik, insbesondere in der Berechenbarkeitstheorie und im Compilerbau, um eindeutig festzulegen, ob ein Wort Element einer Sprache ist und zum anderen, um Eigenschaften dieser formalen Sprachen zu untersuchen bzw. zu beweisen.

Grammatik

Eine **Grammatik** G ist ein 4-Tupel $G = (V, \Sigma, P, S)$, das folgende Bedingungen erfüllt:

V ist eine endliche Menge von **Nicht-Terminalen** bzw. **Variablen**

Σ ist das (endliche) **Alphabet** bzw. die Menge der **Terminal(symbol)e**.

P ist eine endliche Menge von **Regeln** bzw. **Produktionen** mit

$P \subseteq (V \cup \Sigma)^+ \times (V \cup \Sigma)^*$ (mit mindestens einem Element aus V auf der linken Seite)

$S \in V$ ist die **Startvariable** bzw. das **Axiom**.

Erläuterung:

Eine Ableitungsregel ist ein Paar (l, r) (linke und rechte Seite der Regel). Wenn in einem Wort z die linke Seite l einer Produktion (l, r) Teilwort von z ist, darf l durch r ersetzt werden. Wir schreiben auch alternativ: $l \rightarrow r$

Beispiel : Arithmetische Ausdrücke

- Arithmetische Ausdrücke über der Variablen a und den Operationen $+$ und $\cdot$.
- Arithmetische Ausdrücke enthalten zusätzlich Klammern.
- Die Menge der arithmetischen Ausdrücke ist (stark vereinfacht) folgendermaßen aufgebaut.
 - a , $a + a$ und $a \cdot a$ sind arithmetische Ausdrücke.
 - Falls A_1 und A_2 arithmetische Ausdrücke sind, dann sind auch $(A_1 + A_2)$ und $(A_1 \cdot A_2)$ arithmetische Ausdrücke.

Beispiel : Arithmetische Ausdrücke

- Unsere einfachste Form der Menge arithmetischer Ausdrücke lässt sich nun leicht durch eine Grammatik $G = (V, \Sigma, P, S)$ beschreiben.
- Es ist $V = \{S\}$, $\Sigma = \{ (,), a, +, \cdot \}$ und P enthält die folgenden Regeln:
 - $S \rightarrow (S) + (S)$
 - $S \rightarrow (S) \cdot (S)$
 - $S \rightarrow a$
 - $S \rightarrow a + a$
 - $S \rightarrow a \cdot a$
- Alternativ schreiben wir:
 - $S \rightarrow (S) + (S) \mid (S) \cdot (S) \mid a \mid a + a \mid a \cdot a$

Grammatiken

- Man benutzt die Notation $w \rightarrow z$, wenn sich z in einem Schritt (durch Anwendung einer Ableitungsregel) aus w ableiten lässt.
- Die Notation $w \rightarrow^* z$ bedeutet, dass sich z aus w in endlich vielen Schritten ableiten lässt.
- Die von einer Grammatik G erzeugte **Sprache** $L(G)$ besteht aus allen $z \in \Sigma^*$, für die $S \rightarrow^* z$ gilt.

Grammatik und Sprache

- Die von einer Grammatik $G = (V, \Sigma, P, S)$ **erzeugte Sprache** ist

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow_G^* w\}$$

- Menge aller Wörter aus Alphabetsymbolen, die aus der Startvariable S ableitbar sind.
- Zwei Grammatiken G_1 und G_2 heißen äquivalent, genau dann wenn sie die gleiche Sprache erzeugen, wenn also

$$L(G_1) = L(G_2)$$

Beispiel

Die Grammatik $G = (\{S, A, B\}, \{a, b\}, P, S)$ mit folgender Produktionsmenge P :

$$S \rightarrow ABS \mid \varepsilon \quad AB \rightarrow BA \quad BA \rightarrow AB \quad A \rightarrow a \quad B \rightarrow b$$

erzeugt die Sprache:

$$L = \{w \in \{a, b\}^* \mid \#_a(w) = \#_b(w)\}$$

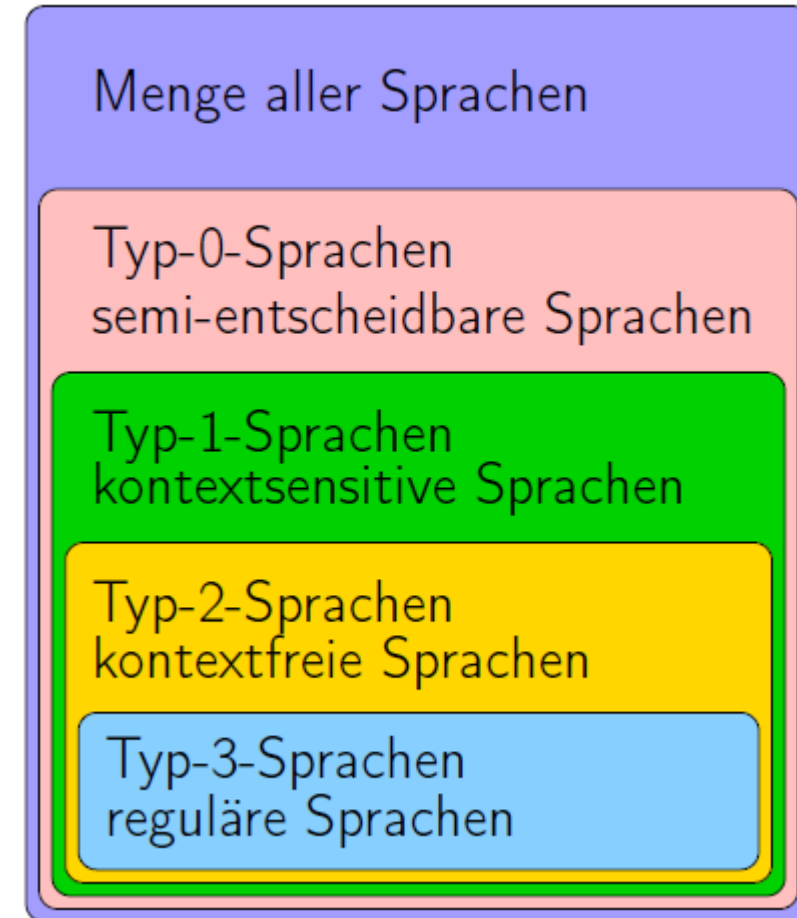
($\#_a(w)$): Anzahl der a 's im Wort w)

Einteilungen

- Grammatiken und die von ihnen erzeugten Sprachen werden in verschiedene Klassen eingeteilt.
- Die Art der erlaubten Produktionsregeln erzeugen eine Hierarchie von Grammatiken.
 - ➔ z.B. Chomsky-Hierarchie

Chomsky-Hierarchie

- **Typ-0-Grammatiken:**
keine Einschränkung
- **Typ-1-Grammatiken:**
Alle Regeln haben die Form: $Q_1 A Q_2 \rightarrow Q_1 B Q_2$ oder $S \rightarrow \epsilon$
wobei S dann nicht auf der rechten Seite auftreten darf
und mit $Q_1, Q_2 \in (V \cup \Sigma)^*$ und $A \in V, B \in (V \cup \Sigma)^+$
- **Typ-2-Grammatiken:**
alle Regeln haben die Form $A \rightarrow v$ mit $A \in V$ und $v \in (V \cup \Sigma)^*$
- **Typ-3-Grammatiken:**
alle Regeln haben die Form $A \rightarrow v$ mit $A \in V$ und $v = \epsilon$
oder $v = aB$ mit $a \in \Sigma$ und $B \in V$ (rechtsregulär)
- Eine Sprache L ist vom Typ i , wenn sie von einer Typ- i -Grammatik erzeugt werden kann.



Chomsky-Hierarchie: Fakten

- Jede Typ- i -Sprache ist auch eine Typ- $(i - 1)$ Sprache.
- Die Typ-0-Sprachen sind genau die rekursiv aufzählbaren Sprachen, werden also von (deterministischen oder nichtdeterministischen) Turingmaschinen erkannt.
- Die Typ-1-Sprachen können von nichtdeterministischen, linear beschränkten Turingmaschinen erkannt werden.
- Es gibt Typ-1.5-Sprachen, die **deterministisch kontextfreien Sprachen**, welche von einem deterministischen Kellerautomaten akzeptiert werden. Sie bilden die Basis für die meisten Programmiersprachen.
- Die Typ-2-Sprachen sind genau die Sprachen, welche von sogenannten nichtdeterministischen **Kellerautomaten** akzeptiert werden. Ihre Grammatiken haben die sogenannte **Greibach-Normalform**. Analog zu Typ-3-Sprachen gibt es hier Parser, welche in linearer Zeit über die Eingabe laufen (zusätzlich zu Faktoren oder Summanden in der Größe der Grammatik).
- Die Typ-3-Sprachen sind genau die regulären Sprachen.

Abgeschlossenheitseigenschaften

	Typ 3 - regulär	Typ 2 - kontextfrei	Typ 1 - kontextsensitiv	Typ 0
Komplement	Ja	Nein	Ja	Nein
Vereinigung	Ja	Ja	Ja	Ja
Schnitt	Ja	Nein	Ja	Nein
Differenz	Ja	Nein	Ja	Nein
Konkatenation	Ja	Ja	Ja	Ja
Kleenestern	Ja	Ja	Ja	Ja
Umdrehen	Ja	Ja	Ja	Nein

Entscheidbarkeiten

	Typ 3 - regulär	Typ 2 - kontextfrei	Typ 1 - kontextsensitiv	Typ 0
Leere	Ja	Ja	Ja	Nein
Universalität	Ja	Nein	Nein	Nein
Element in	Ja	Ja	Ja	Nein
Äquivalenz	Ja	Nein	Nein	Nein
Disjunktheit	Ja	Nein	Nein	Nein
Teilmenge	Ja	Nein	Nein	Nein
Regularität	Ja	Nein	Nein	Nein

Ein System ist universell, wenn es alle anderen Systeme seiner Art simulieren kann.

Reguläre Ausdrücke über einem vorgegebenen Zeichenvorrat Σ basieren auf genau drei Operationen: Alternative, Verkettung und Wiederholung.

Anwendungen

- Wofür kann ich dies gebrauchen?
- Sprachparser
 - Compilerbau
Erster Schritt des Compile-Vorgangs ist die Syntaxprüfung des Source-Codes. Entspricht der Source-Code den Vorgaben der Programmiersprache?
 - XML-Datensätze
XML-Dokumente sind die Wörter einer XML-Sprache. Auch hier muss ein Parser die Korrektheit der Syntax überprüfen. Also: Ist das gegebene Dokument Element der vorgegebenen XML-Sprache?

MiniJavaScript

- Grammatik einer einfachen Script-Sprache

```

Programm → Befehle
Befehle → Befehl | Befehl ; Befehle
Befehl → Variable = Ausdruck
          | Variable = prompt ( " " , " " )
          | write ( Ausdruck )
          | write ( Zeichenkette )
          | writeln ( Ausdruck )
          | writeln ( Zeichenkette )
          | if ( Kondition ) { Befehle }
          | while ( Kondition ) { Befehle }
    
```

```

Kondition → Ausdruck == Ausdruck
            | Ausdruck < Ausdruck
            | Ausdruck > Ausdruck
Ausdruck → Ausdruck + Ausdruck
            | Ausdruck - Ausdruck
            | Variable
            | Zahl
            | - Zahl
Variable → A | B | C
Zahl → Ziffer | Ziffer Zahl
Ziffer → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
Zeichenkette → " Alle-ASCII-Zeichen-außer-" "
    
```

Berechnungsmodelle

- Was bedeutet es überhaupt im Allgemeinen, dass eine **Funktion berechenbar** oder eine **Sprache akzeptierbar** ist?
- Wie kann man ein **generelles Berechnungsmodell** definieren?
- Was ist das **Maschinenmodell für Chomsky Typ 0-Sprachen**?

Berechnungsmodelle

- Im Laufe der Zeit sind mehrere Berechnungsmodelle entwickelt worden, die jedoch alle zueinander äquivalent sind:
 - Turing-Maschinen (benannt nach Alan Turing)
 - While-Programme, Goto-Programme
 - μ -rekursive Funktionen
- Turing-Maschinen mit entsprechender Beschränkung dienen auch als Maschinenmodell für Typ-1 Sprachen

Unentscheidbarkeit

Gibt es Sprachen, für die das **Wortproblem** ($w \in L?$) **nicht entscheidbar** ist?

Antwort: ja

Kann man Funktionen definieren, die **nicht berechenbar** sind? Das heißt, es gibt keine Turing-Maschine/kein While-Programm/keine μ -rekursive Funktion, die diese Funktion berechnet?

Antwort: ja

Außerdem: Wie zeigt man diese negativen Resultate?